17-214/514: Agentic Software Development

Coordinating and Communicating Systems

Hammad Ahmad and Claire Le Goues • Monday, November 9, 2026 • Lecture

Administrivia

- Survived the midterm?
- A5 is due next Monday.

What did we talk about last time?

- Concurrency can come with users, background work, and networks, whether or not our code starts a thread.
- Check-then-act and read-modify-write on shared mutable state are two common races. A test usually sees one interleaving.
- Remedies (locks, database constraints and conditional updates, immutability, confinement) make the check and the act atomic, or avoid shared mutable state. Too much locking can slow things down, and two locks can deadlock.
- Async/await is a common way to write concurrent code. Await (or handle) every promise, and treat each `await` as a point where another request can run.
- A lock guards only the threads that take the same lock. For concurrent code, have the agent list the shared state and its guards, and force the interleaving you're worried about.
- We left one problem for today (**half-done work**). The booking was written, the notification wasn't, and the process crashed in between.

Learning goals for today

- Explain how latency, lost messages, and partial failure change calls between services
- Choose a synchronous call, a queue, or an event for an operation, and defend the choice
- Use a transactional outbox to send a message reliably after a database write
- Use a decorator to add behavior (e.g., retries) while keeping the interface
- Design handlers that are safe to run twice, and explain effectively-once processing
- Test coordination code under duplicates, reordering, and crashes

Three emails for one booking



Each part did what it was designed to do, but the user still got three emails.

Services over a network

Threads vs. services

Last time
Two threads in one process, sharing memory. A lock or a database constraint made the check and the act atomic.

This week
Two **services** on two machines, with a network between them. Each has its own memory. A lock in one process doesn't guard the other.

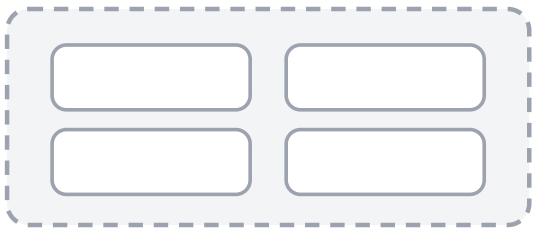
Things still interleave, but now with a network in the middle.

Network calls can cause headaches

- **Latency:** A function call takes nanoseconds, but a call to another machine usually takes milliseconds.
- **Loss:** A request or its reply can get lost on the way.
- **No shared memory:** A service knows only what it has been told. Information it needs arrives as messages.

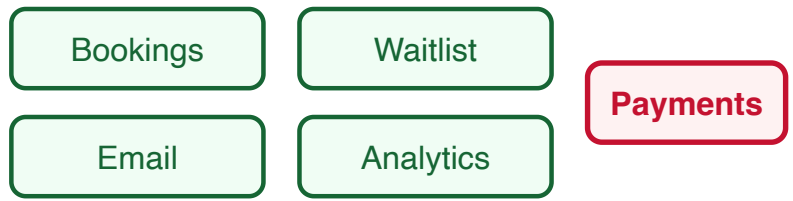
Partial failure in distributed systems

One process



If it crashes, all of it stops.

Five services



Payments is down. The other four keep running.

> A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.
>
> - Leslie Lamport, 1987

Did they get the message?

- We call the payment service, and the call times out. Did the request never arrive, or did the payment go through and only the reply get lost?
- From our side, the two look the same. Over a network that can lose messages, no protocol can make both sides sure they agree (the **two generals problem**).
- After a timeout, we have to decide whether to retry without knowing if the payment went through.

The request was lost



Retrying is right. Nothing was charged.

The reply was lost



Retrying charges the card twice.

Both look like a timeout from our side.

Consistency or availability during a partition

- A **network partition** is when the network between two groups of machines fails, but both groups keep running and serving users.
- During a partition, a system has to pick one of two (the **CAP theorem**):
  - **Availability:** Both sides keep answering, but their answers may disagree.
  - **Consistency:** Answers stay in agreement, but some requests are refused until the network heals.
- You can make a different choice for different data.
  - For example, bookings might stop during a partition (so that two sides can't book one room twice), while room descriptions are still served.

> It was clear through querying the state of the database clusters that we needed to stop running jobs that write metadata about things like pushes. We made an explicit choice to partially degrade site usability by pausing webhook delivery and GitHub Pages builds instead of jeopardizing data we had already received from users. In other words, our strategy was to prioritize data integrity over site usability and time to recovery.

GitHub, 2018, after a 43-second partition left the two coasts with different writes

Ways to talk

Synchronous (sync) calls

- Service A calls service B and **waits** for the answer (like a function call that crosses the network).
  - For example, the app calls the booking service and waits for "confirmed" or "room taken" before showing the student anything.
- It's simple to write and to reason about, since the answer comes back in the same request.
- But A now waits as long as B takes, and usually fails when B fails.
  - In a chain of calls, one slow or failed service can slow or fail the whole request.

Asynchronous calls

- Hand the work off, answer the caller now, and let the result arrive later.
  - For example, the booking response doesn't have to wait for the confirmation email.
- But, "it's done" is now "it's **in progress somewhere**". The caller needs a way to learn the result or the failure afterwards.
  - (Recall that a promise is a value that arrives later. With an asynchronous message to another service, the result also arrives later.)

Queues and workers

- A producer puts work into a queue, and consumers take it out. Neither waits for the other.
  - For example, the booking service puts "email the confirmation for booking 7" on a queue, and an email worker sends it when it gets to it.
  - (We saw this last time inside one process, as the work queue of a thread pool.)
- The producer and the consumers don't need to be running at the same time.
- A queue can absorb bursts. During a booking rush, work waits in the queue. Workers can take it at their own pace.
- When the queue backs up, you can add workers. Each worker takes one job, works on it alone, and then takes another.
  - (This is also a kind of confinement. Usually no two workers have the same job at once, so the job's own data needs no lock. Database constraints may still be needed if two jobs touch the same table.)

Backpressure

- Producers can outrun consumers (e.g., during exam week). An unbounded queue can then grow until it runs out of memory.
- Adding workers helps, but only up to a limit.
  - For example, do they all share a database?
- **Backpressure** involves the slower side pushing back. Bound the queue, then choose what happens when it's full:
  - Block the producer (everyone slows down).
  - Reject with a clear error (shed load).
  - Drop the message (only for data you can afford to lose).
- A rate limit that answers "429 Too Many Requests" is a kind of backpressure too.
  - A5's notification service does this, BTW.

Streams

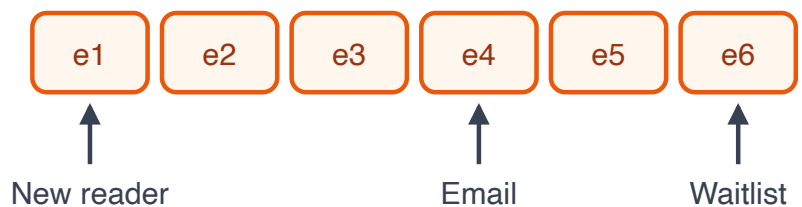
- A **stream** is an append-only log of messages that stays around. Each consumer reads at its own pace, from any point, and can read again.
  - For example, every `BookingMade` goes into a stream, so a new analytics service can read past bookings from the start (as far back as the stream keeps them).
- Recall the event log, which keeps every event for history and replay. A stream is similar, but shared between services. And it usually keeps messages only for a set time.

Queue



Each job goes to one worker,
and then it leaves the queue.

Stream



Each reader keeps its own position.

Publish/subscribe

- Publishers send events to a **message broker**, and subscribers receive the ones they subscribed to. Publishers and subscribers don't know about each other.
- This is Observer across a network.
- The broker usually also stores messages and tracks who has received them. Each broker promises a particular delivery guarantee.
  - If you use one, make sure you check what guarantees yours gives.

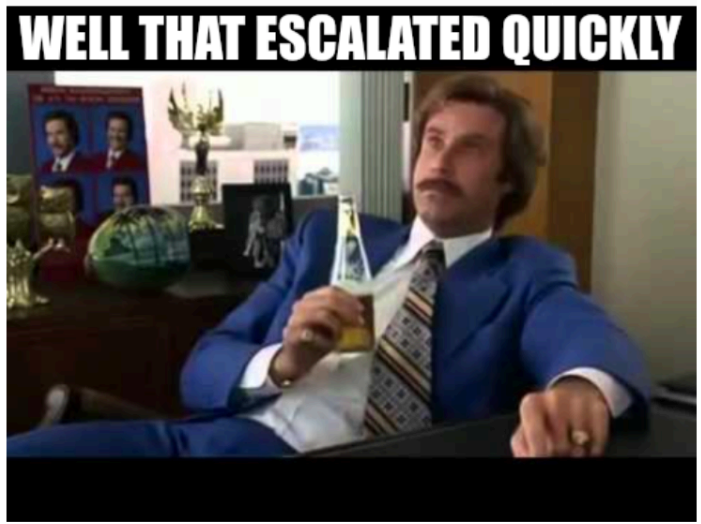
```
// Reminder: Observer inside one process (from the design patterns lecture)
bookings.on(BookingCancelled.class, e -> waitlist.promoteFor(e.slot()));
bookings.on(BookingCancelled.class, e -> analytics.record("cancellation", e.booking()));
```

Delivery guarantees

At-most-once: send and don't retry
No duplicates, but messages can be lost. It can make sense when the next message replaces a lost one (e.g., a room's occupancy sensor reporting every minute), but not for payments.

At-least-once: retry until acknowledged
No message is lost, but **duplicates can arrive**. This is how one booking got three emails earlier.

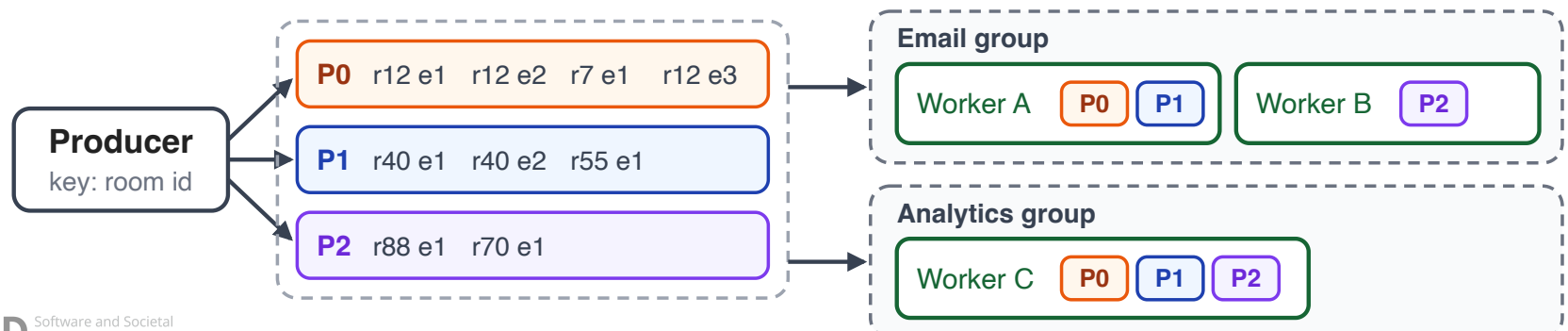
- There is a "retries" setting
  - defaults to 0 for Kafka <= 2.0
  - defaults to 2147483647 for Kafka >= 2.1



ProgrammerHumor.io

Partitions and consumer groups

- Many brokers that keep streams (e.g., Apache Kafka) split each stream into **partitions** (smaller logs). Messages with the same key (e.g., the room id) go to the same partition and stay in order.
- A **consumer group** is a set of workers that read one log together. Each partition goes to one worker in the group. Adding workers helps until each worker reads one partition.
  - Within a group, the stream works like a queue. Across groups, it works like pub/sub (e.g., email and analytics as two groups).
- Order is kept only within a partition (room 12's and room 40's events can interleave).



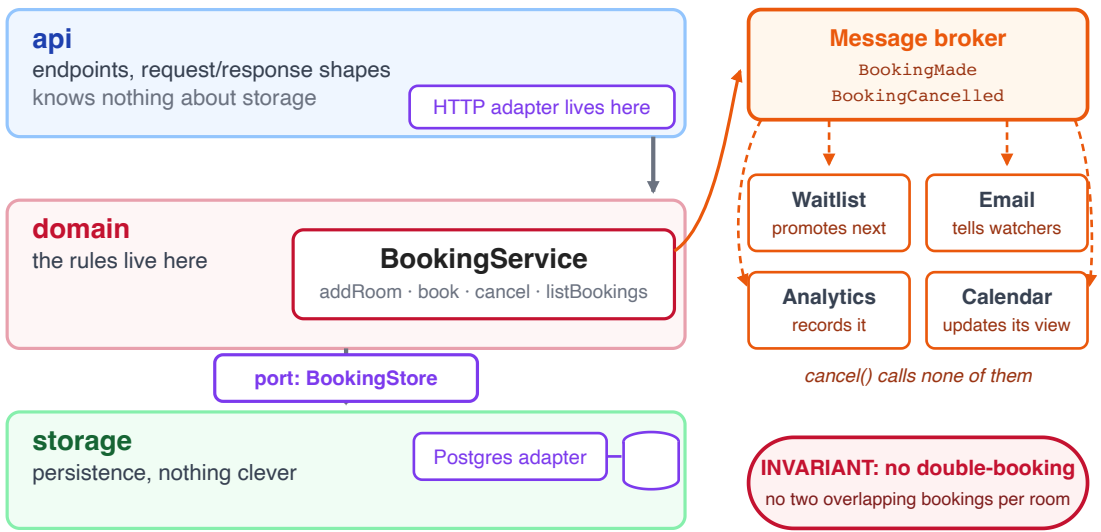
Thinking in events

Events

- An **event** records something that already happened. Its name should be in the past tense (`BookingMade` , `BookingCancelled`).
- In an event-driven design, services publish the events they produce and react to the ones they care about. The publisher doesn't know who reacts.
  - For example, accounts announces `TierChanged` , and bookings updates that student's booking limit.
- Not everything should be an event. A **query** ("is room 12 free?") and a **command** ("book room 12") are requests that need an answer. The event (`BookingMade`) should be published only after the command succeeds.

One cancellation reaching four subscribers

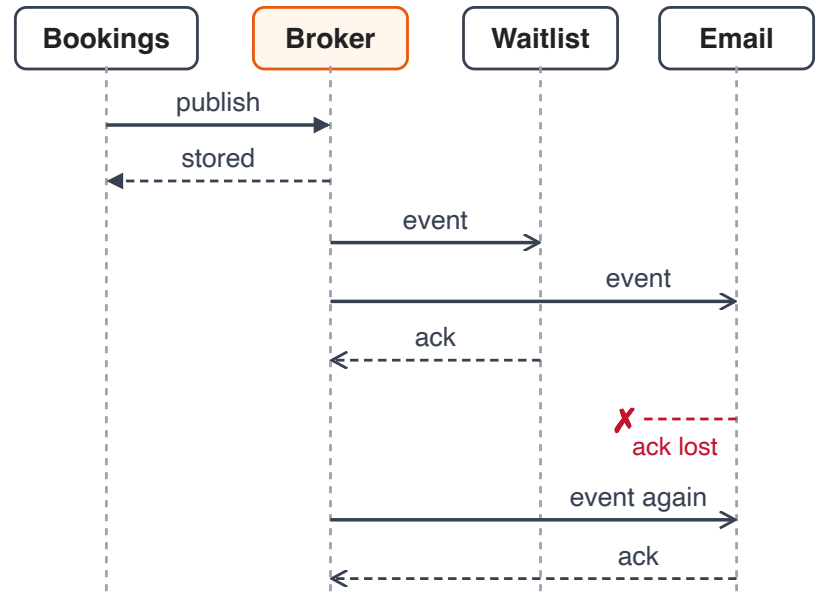
`BookingCancelled` for booking 7 is published once. Each subscriber reacts on its own.



A5 is similar. One message posted to a channel can mean notifications for many of its members.

Reading the cancellation as a sequence diagram

- Recall that object interaction diagrams show who calls whom and in what order. This one crosses services.
- Open arrowheads mark asynchronous messages. The broker doesn't wait for a subscriber to finish.
  - (Dashed lines are replies.)
- Email gets the event twice, since its first acknowledgment (ack) was lost.
- The diagram doesn't say whether Waitlist or Email runs first.



As an aside: CQRS

- **Command-query separation** says a method should either change state (a command) or return an answer (a query), but not both.
  - A test can call a side-effect-free query as often as it likes, which makes the code easier to observe. A query is also safe to retry.
- **CQRS** (command query responsibility segregation) applies the same split to a whole system. Commands go to the write side (the bookings database). Queries can be answered from read models built from the events.
  - For example, a subscriber to `BookingMade` and `BookingCancelled` can keep a "free rooms this week" view.
  - The cost is that read models can lag behind writes, and there's more to run.

Sync call, queue, or event?

It depends.

On what?

Coupling

Should the caller know the other service exists? A sync call couples the most. An event couples the least.

Waiting

Does the user need the answer **in this response**? Room availability, yes. A confirmation email, usually no.

Failure

Should the caller fail when the other service is down? With a sync call, it usually does.

History

Does anyone need a record of what happened? A stream keeps one, but a sync call doesn't.

Let's practice: sync call, queue, or event?

With a neighbor, pick one for each operation and explain why (coupling, waiting, failure, or history).

1. Checking the student's tier (from accounts) against the booking limit.
2. Building the semester usage report a department asks for. It reads every booking.
3. Keeping a record of every booking change for the university's compliance office.
4. Cleaning up holds that expired overnight.

Canvas → Quizzes → Lecture 18 In-Class Quiz

Reliable delivery

Write to the database, then publish

```
def book(booking):
    with db.transaction():
        db.insert("bookings", booking)
    broker.publish(BookingMade(booking))   # a crash here loses the event
```

- These are two writes to two systems. Usually no transaction can cover both.
- A crash between them loses the event. Publishing first instead can announce a booking that then fails to commit to the database.
- (This is the half-done work we left for today.)

The transactional outbox

```
def book(booking):
    with db.transaction():
        db.insert("bookings", booking)
        db.insert("outbox", BookingMade(booking))   # same transaction

def relay():                                # runs on its own, in a loop
    for msg in db.unsent("outbox"):
        broker.publish(msg)                 # or call the email service directly
        db.mark_sent("outbox", msg.id)
```

- With a **transactional outbox**, the booking and its message commit together, or neither does.
- A **relay** sends what's in the table. A crash after sending but before marking means it sends again, so delivery is at least once.
- The outbox table can itself be the queue. A worker can consume it without needing a broker.

Don’t add a broker unless you need it

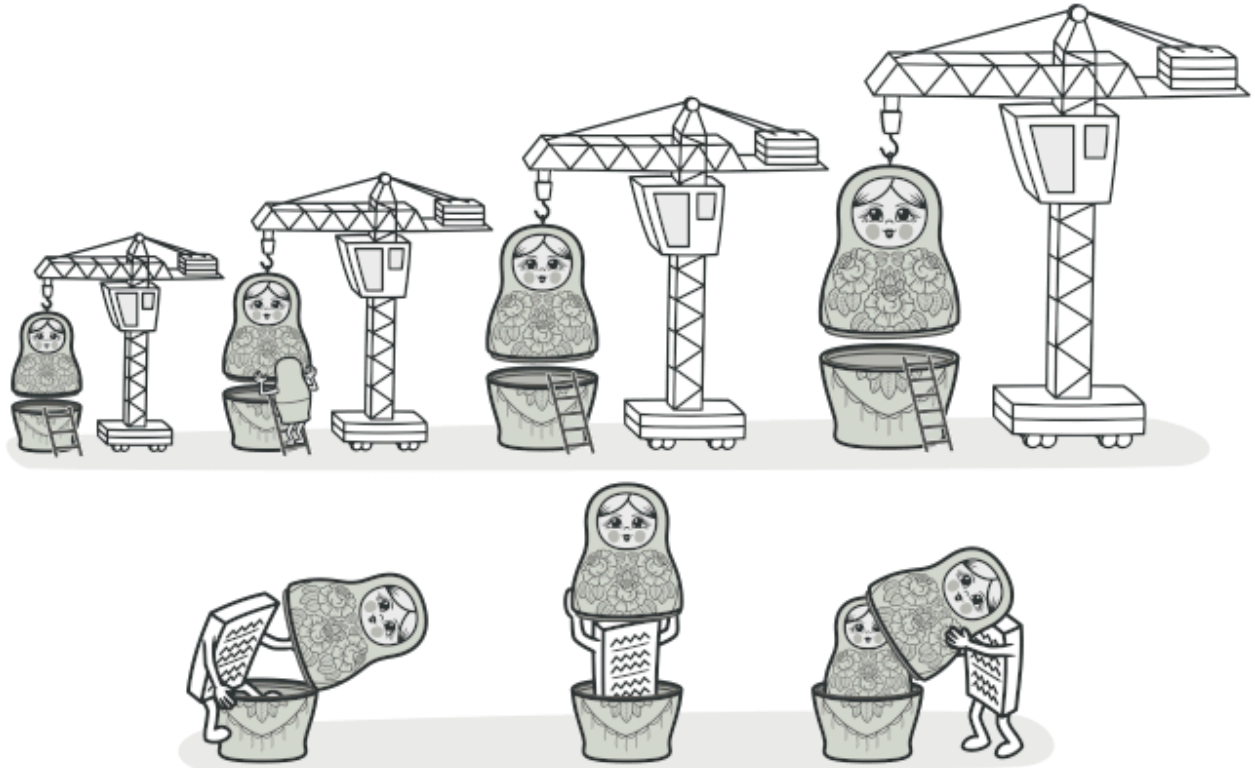
- A broker is another process to run, monitor, and upgrade. It’s worth it when several subscribers read the same events, or need to replay them.
- An outbox uses the database you already have. It costs an outbox table, an extra write per message, and a relay that watches that table (e.g., by polling it or by reading the database’s change log).
- The two can also combine. The outbox makes sure the message gets to the broker, while the broker delivers it to every subscriber.
- A5 leaves the delivery path to you (a broker, an outbox, or something else).

Retries can duplicate

- We retry a call that timed out. If the first request had gone through, there are now two.
- We can't tell which happened (the two generals problem). Giving up on every timeout would lose real work, so retrying is usually still right.
- Retry with growing waits (**exponential backoff**, e.g., 1s, 2s, 4s), and stop on errors a retry won't fix (e.g., a 403 or a 404).
- With at-least-once delivery, the same message can arrive more than once, so handlers have to cope with duplicates.

> In 15.10.2 we did Update mail gem to v2.8.1. Something in the line of that dependency seems to be causing an issue with outgoing emails for some customers. It appears that the Sidekiq job doesn't register the successful sending for some reason, and performs all three retries of the job, resulting in four sent emails total. (Similar behavior was observed in #231353 (closed) a while ago.)

GitLab, issue 410336 (2023)



Refactoring.Guru

Decorator

The problem: We want retries around sending email, without changing the email sender or the code that calls it.

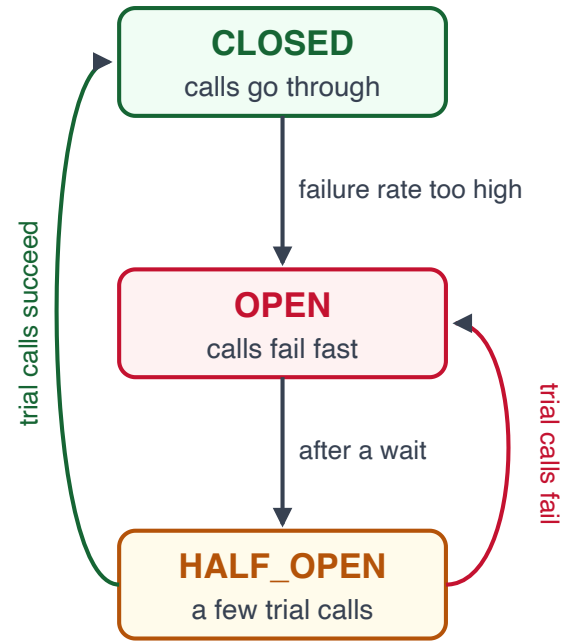
```
class RetryingEmailSender implements EmailSender {
  private final EmailSender inner;
  RetryingEmailSender(EmailSender inner) {
    this.inner = inner;
  }

  public void send(Email email) {
    for (int attempt = 1; ; attempt++) {
      try { inner.send(email); return; }
      catch (SendTimeout e) {
        if (attempt == 3) throw e;
        backOff(attempt);   // wait 1s, then 2s
      }
    }
  }
}
```

- A **decorator** wraps an object, keeps its interface, and adds behavior.
- Callers still see an `EmailSender`, so they don't change.
  - But the retry is hidden from callers. If a timed-out send went through, the email goes out twice.
- When attempts run out, a decorator can also fall back to a cached or degraded answer instead of throwing an exception.

Circuit breakers

- A **circuit breaker** wraps the calls to another service. While calls succeed, they go through.
- When too many calls fail, it opens and stops calling the service for a while. Calls fail fast instead of waiting on a service that's down.
- After a wait, it lets a few trial calls through to check whether the service is back.
- A retry decorator can wrap the breaker. While the breaker is open, each retry fails fast.
- Libraries like resilience4j provide retries and circuit breakers as decorators you stack around a call.



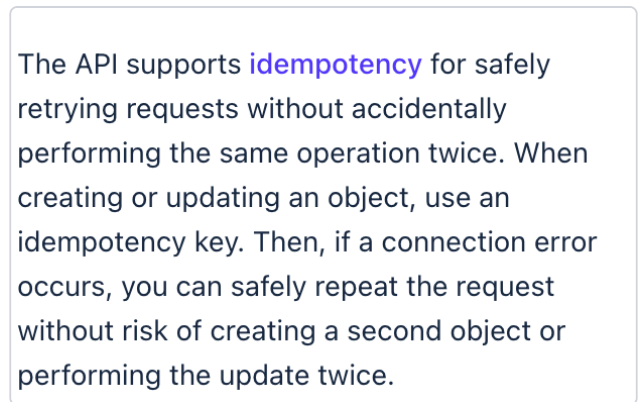
resilience4j, CircuitBreaker docs

Idempotence

- An operation is **idempotent** when doing it twice has the same effect as doing it once.
  - For example, cancelling booking 7 again leaves it cancelled.
  - Charging a card $30 isn't idempotent (twice is $60).
- Payment APIs often take an **idempotency key**. A retry with the same key doesn't charge again.
  - (This makes the timed-out payment from earlier safe to retry.)
- Handlers can be made idempotent on purpose, so that duplicates do no harm.



ProgrammerHumor.io



Stripe, API reference

A handler that is safe to replay

```
def handle_booking_made(event):
    # idempotency key: has this event been processed?
    if db.exists("processed_events", event.id):
        return                          # seen before: acknowledge and skip
    with db.transaction():
        db.increment("rewards", event.user_id, 10)   # same database
        db.insert("processed_events", event.id)
```

- The event id is the idempotency key. Checking the processed set makes a replay do nothing.
- The work and recording the id happen in **one transaction**. Otherwise a crash in between brings the duplicate back.
- Adding 10 points twice would give 20. Where you can, make the operation itself idempotent instead (e.g., setting `status = 'paid'` twice is fine).

Can we get exactly-once delivery?

> There are only two hard problems in distributed systems:
> 2. Exactly-once delivery
> 1. Guaranteed order of messages
> 2. Exactly-once delivery
>
> - Mathias Verraes, 2015

- Over a network that can lose messages, delivery is at most once or at least once.
- Products that advertise exactly-once (e.g., Kafka) mean it for work that stays inside them, like reading a message and writing the result back to the broker. A subscriber's own side effects, like an email, aren't covered.
- At-least-once delivery plus idempotent handlers gives **effectively-once** processing.

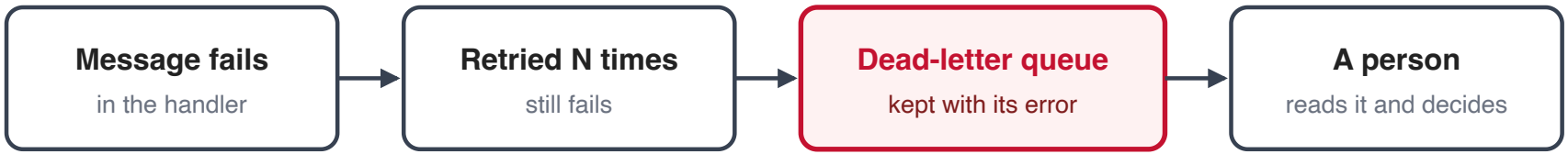
Message order

- `BookingCancelled` can arrive before the `BookingMade` it cancels. Messages take different routes and retries, so arrival order can differ from send order.
- When order matters, you can partition by entity (all of room 12's events in one partition), or number the messages and let the consumer reorder or reject them.
  - A partition keeps the order the broker received. With several producers, that can differ from the order things happened, which is where numbering helps more.



ProgrammerHumor.io

Dead letters



- Retrying forever can let one bad message (malformed, or one that triggers a bug) block the messages behind it. Moving it aside lets the rest go through.
- A dead-letter queue should have someone responsible for reading it and deciding what happens to each message (fix and redeliver, or drop).

Sagas

- Recall that a transaction usually can't span the databases of two services. "Reserve the room and charge the card" becomes two steps that can fail separately.
- A **saga** runs a sequence of local steps. Each step has a **compensating action** that undoes it in business terms.



- A compensation is recorded as a new step (like an accountant's correcting entry). The reservation stays in the history.
  - If a compensation fails, it is usually retried (so it has to be idempotent too). If it keeps failing, it can go to a dead-letter queue.

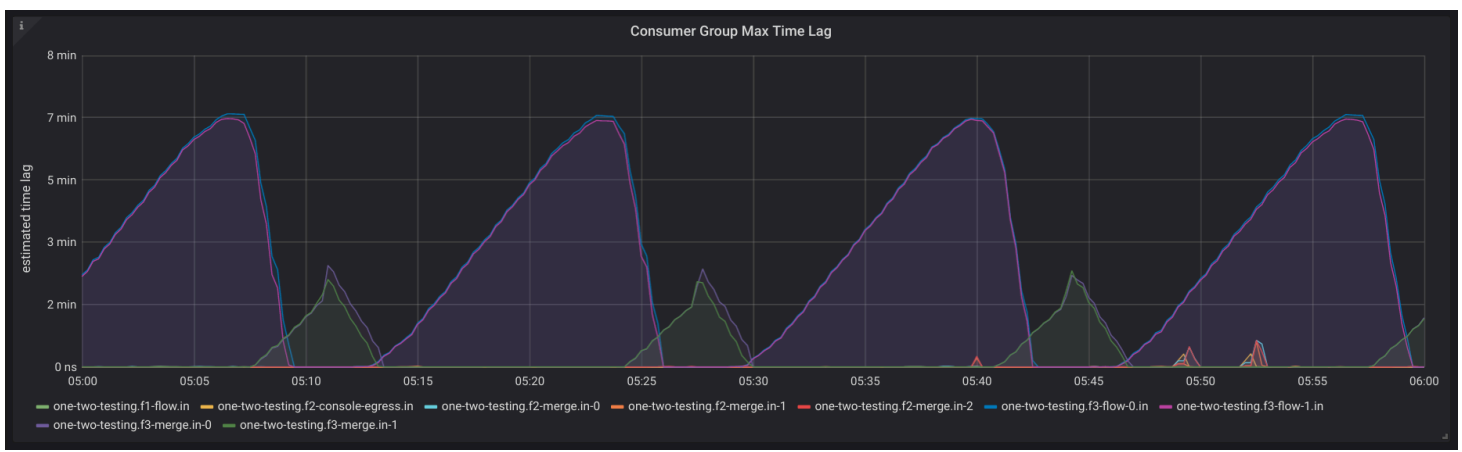
A crash in the middle of a saga

```
1. reserve room          done
2. charge card           done
3. confirm booking       CRASH
```

- After a restart, the room is held and the card is charged, but the booking isn't confirmed. It's half-done work again (now with money in it).
- The saga should record in storage which steps finished, so the restarted worker can resume at step 3.
- But the crash may have come after the booking was confirmed and before the saga recorded it. Then the restart confirms it again, so step 3 has to be idempotent.

Consumer lag

- **Lag** is how far behind the consumers are (in messages or in time).
- Growing lag can mean dying workers, a message that keeps failing, or more load than the workers can handle.
- It often shows a problem before users report it, which makes it a good number for a dashboard.



github.com/seglo, Kafka Lag Exporter

Verifying coordination code

Checking generated coordination code

- Coordination bugs usually show up only under loss, duplicates, reordering, and crashes. A test where every message arrives once, in order, with no crash, passes whether or not the code handles them.
- An agent can quickly write a plausible queue consumer or retry decorator that passes that kind of test.
- For each handler, have the agent write a test for each of three questions. What if this message arrives twice? Out of order? What if we crash after this line?
  - For a crash, stop a worker partway through a saga, restart it, and check that no room is double-booked and no card is charged without a booking.
  - **Chaos engineering** does this to a production system on purpose (Netflix popularized it in the early 2010s with Chaos Monkey).

Practice at home: is this handler safe to run twice?

The agent reports: "Made the confirmation handler idempotent. It records each event id and skips duplicates."

```
def handle_booking_made(event):
    if db.exists("processed_events", event.id):
        return
    email.send(confirmation_for(event.booking_id))
    db.insert("processed_events", event.id)
```

- What if the process crashes right after `email.send`, and the broker delivers the message again?
- Would moving the insert above the send fix it?
- Would you accept the change?

More on A5

- Your `NOTIFICATIONS.md` draft can change after today. If it does, say why in your notifications record.
- The record should also defend how notifications get from your replicas to the notification service (a broker, an outbox, or something else), and what happens to pending ones when a replica stops.
- The scenario test sends bursts across your replicas and stops one partway through. It checks the contract's guarantees, the DM guarantee, and the rate limit.
  - Your harness can do the same to your system.
- Where a decision record no longer holds with replicas, supersede it.

Summary

- A network adds latency, lost messages, and **partial failure**. After a timeout, we can't know whether the other side got the request.
- During a partition, a system can stay consistent or stay available (CAP).
- A sync call is simple but couples the caller to the service it calls. With a queue, the caller doesn't wait, and a full queue needs backpressure.
- Streams can be replayed. Events let subscribers react without the publisher knowing them.
- A transactional outbox commits a message with the data, and a relay sends it at least once.
- Retries can create duplicates, so handlers should be idempotent. At-least-once delivery plus idempotent handlers gives effectively-once processing.
- A decorator (e.g., a retry wrapper) keeps an interface and adds behavior.
- Order holds per partition, failing messages can go to a dead-letter queue, and multi-step work across services can be a saga with compensating actions.
- Coordination code should have tests that deliver twice, reorder, and crash partway through.

What's next

W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
A1 out		A1 due		A2 due · M1		A3 due		A4 due	M2 · A5 chkpt		A5 due		A6 due

- **Wednesday:** Containers and the cloud (where these services run).
- **Friday, Lab 11:** Events versus synchronous calls, and making a handler safe to replay.
- **Monday, Nov 16:** A5 due at 11:59 pm.